# Interactive Tree Simulator

Exploring Binary Search Trees & Visualizing Data Structures for Assignment

CO3_AT2

Done By
S.Jaxon Rizal(192511421)

# Introduction to Tree Simulators

## What is it?

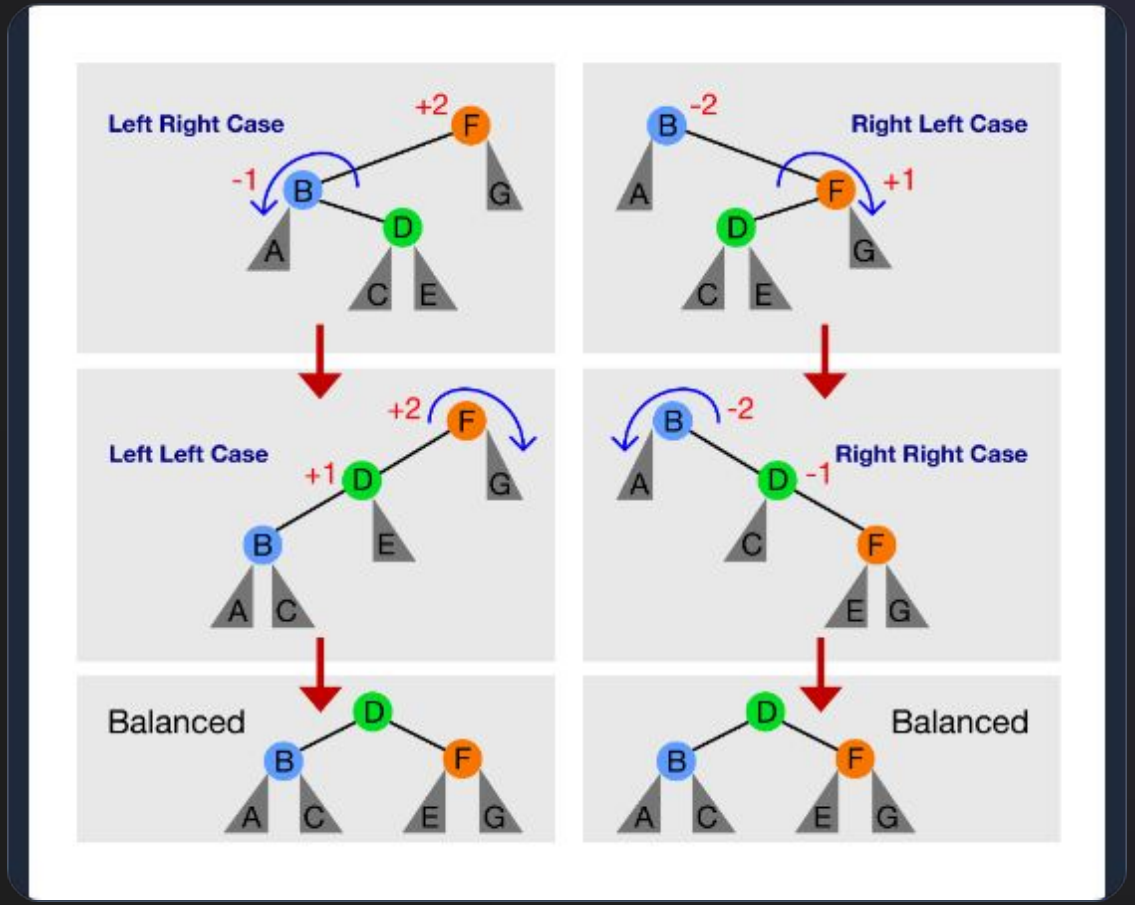
Tree simulators are powerful educational tools that provide real-time visual representations of how hierarchical data structures, like Binary Search Trees (BSTs) or AVL trees, are constructed and manipulated in memory.

## Why use them?

They transform abstract algorithmic concepts into concrete visual sequences. By animating insertions, deletions, and complex traversals, simulators bridge the gap between theoretical code and practical understanding.

# Tool Spotlight: VisuAlgo

- **Overview:** VisuAlgo is a premier platform for rendering animated Binary Search Trees and self-balancing AVL trees, offering a clear window into data structure operations.
- **Custom Inputs:** Allows users to insert custom, comma-separated values to observe how specific datasets form unique tree shapes.
- **Step-by-Step Tracking:** Operations can be paused and stepped through sequentially, showing the exact comparison logic at each node.
- **Visual Balancing:** Clearly demonstrates the complex rotation algorithms (Left-Left, Right-Right) used by AVL trees to maintain balance after insertions.



# Simulation in Action: Inserting Data

## The Scenario

Consider inserting the sequence: [50, 30, 70, 20, 40]. The first value, 50, is established as the Root node, forming the base of our hierarchy.

## The Logic

The simulator visually demonstrates the core BST rule: If a new value is less than the current node, it moves left. If it is greater, it moves right.

## The Result

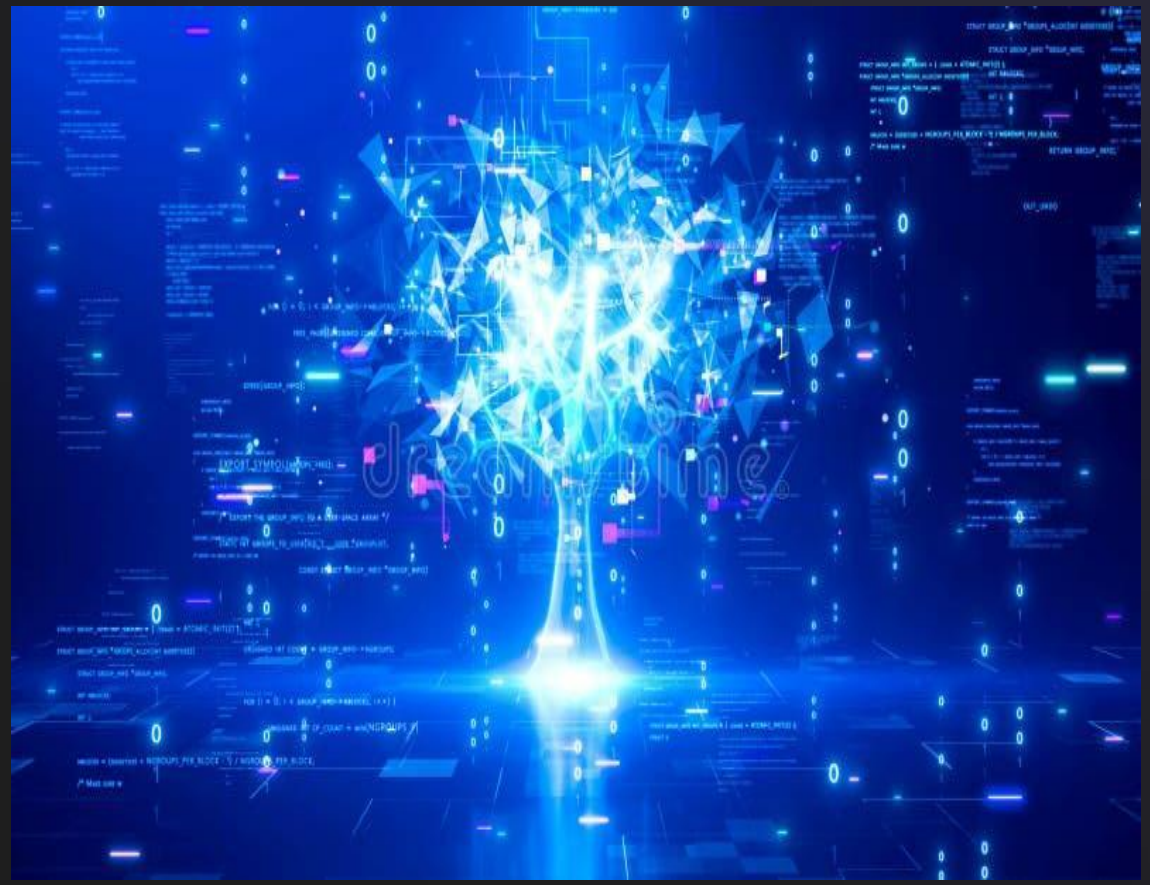
Value 30 goes left of 50. Value 70 goes right. Value 20 compares to 50 (left), then 30 (left again). The visualizer traces this path in real-time.

# Innovation: The Logic Behind Visuals

While simulators render friendly circles and lines, it's crucial to understand the underlying programming reality.

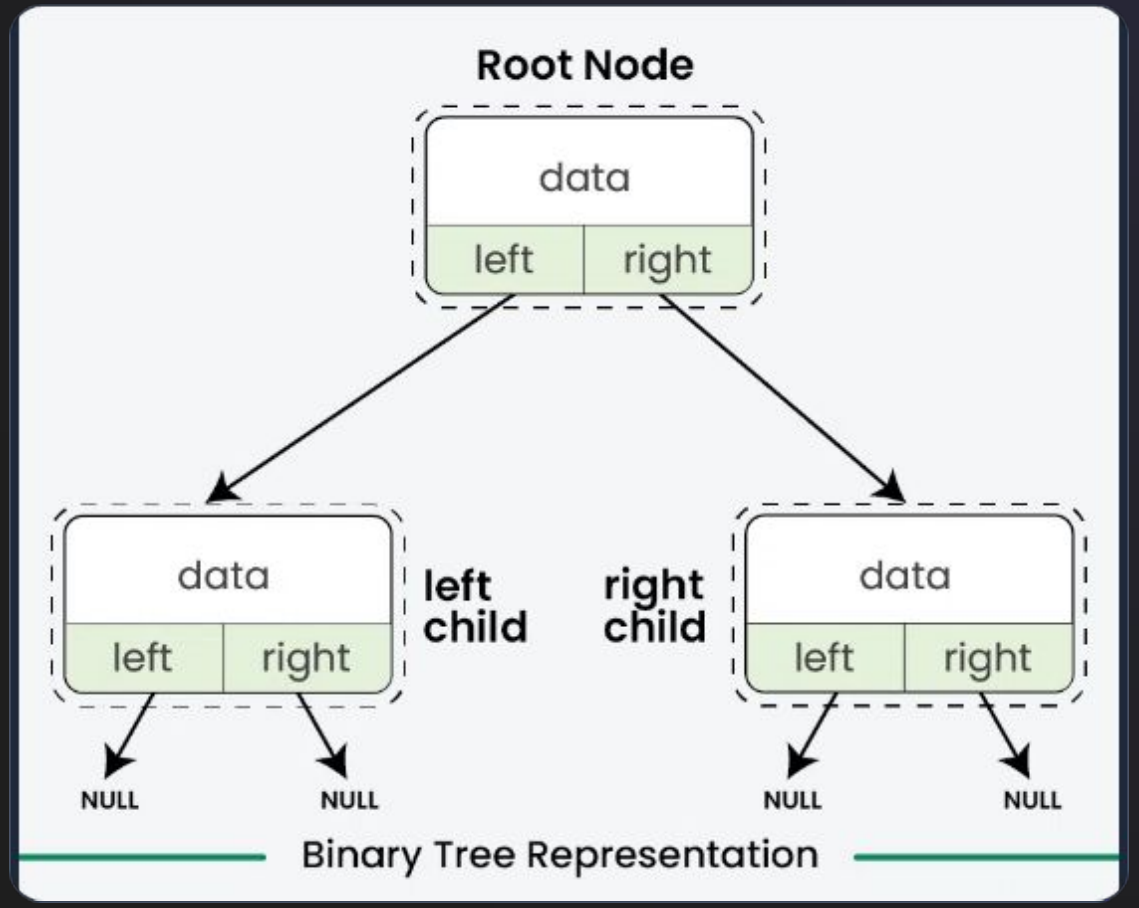
A tree isn't a continuous drawing; it's a collection of distinct memory blocks (nodes) scattered in RAM. The 'lines' are merely pointers—memory addresses connecting one node struct to another.

The visualizer is essentially running a graphical overlay on top of standard pointer traversal logic.



# The Anatomy of a Node in Memory

- **Data Payload:** The core integer, string, or object being managed by the node.
- **Left Pointer (*left):** A memory address pointing to the left child node (values strictly less than parent).
- **Right Pointer (*right):** A memory address pointing to the right child node (values greater than parent).
- **Null Termination:** When a node lacks children (a leaf), these pointers are set to NULL, preventing infinite loops during traversal.



# Alternatives & Applications

## Virtual Labs IIITH

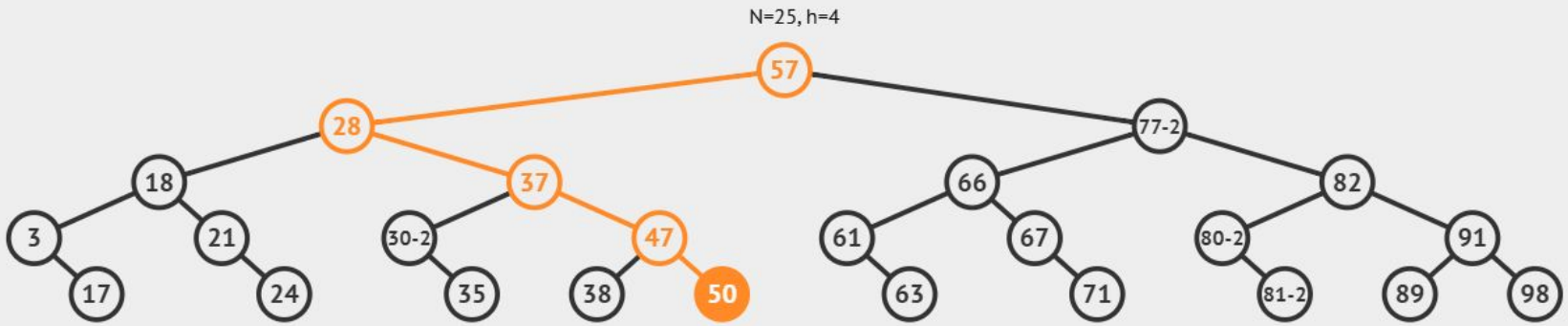
Offers a dedicated interactive practice module where users must manually click through nodes in the correct order to simulate a traversal.

## Database Indexing

In the real world, structures like B-Trees (advanced BSTs) power database indexes, allowing massive systems to find single records in milliseconds.

## Network Routing

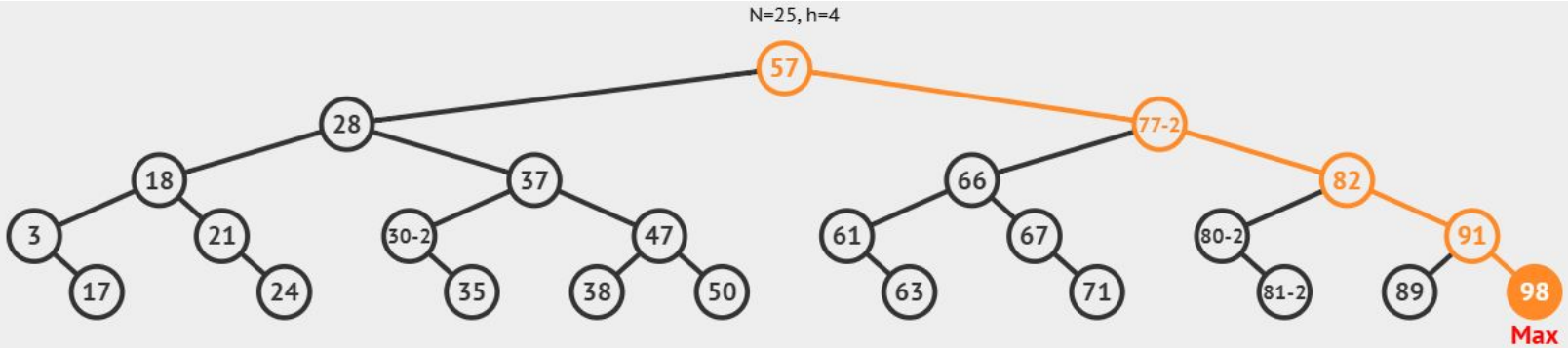
Routing algorithms often utilize tree structures to calculate the most efficient paths across the internet, ensuring data packets don't get lost in loops.




**Search(50)**

Value 50 is found.

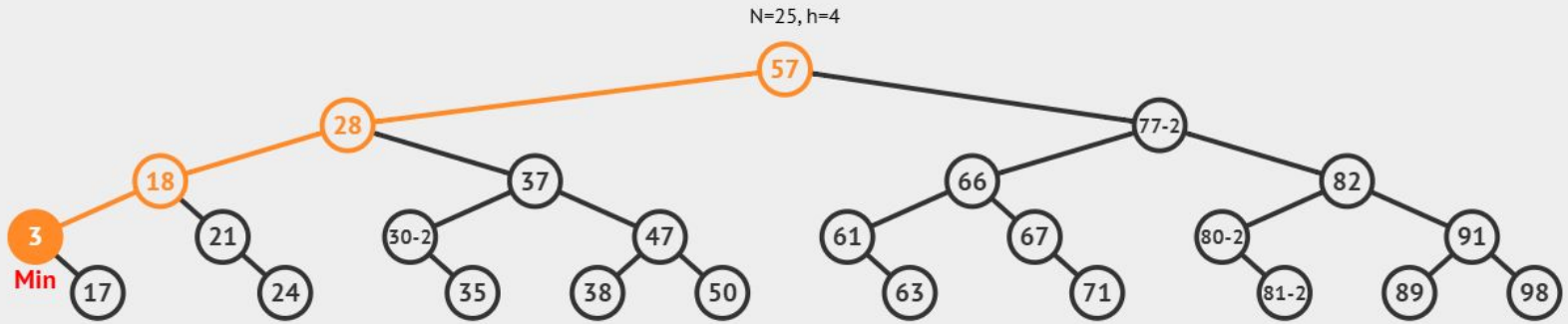
```
if this == null
  return null
else if this key == search value
  return this
else if this key < search value
  search right
else search left
```




## Search(Max)

Maximum value 98 found!

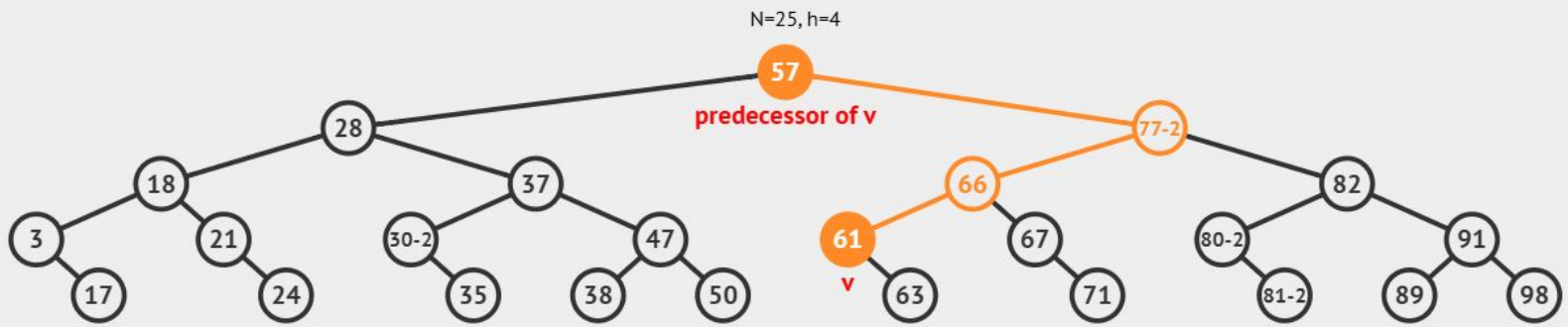
```
if this is null return empty
if right != null
  go right
else return this key
```




## Search(Min)

Minimum value 3 found!

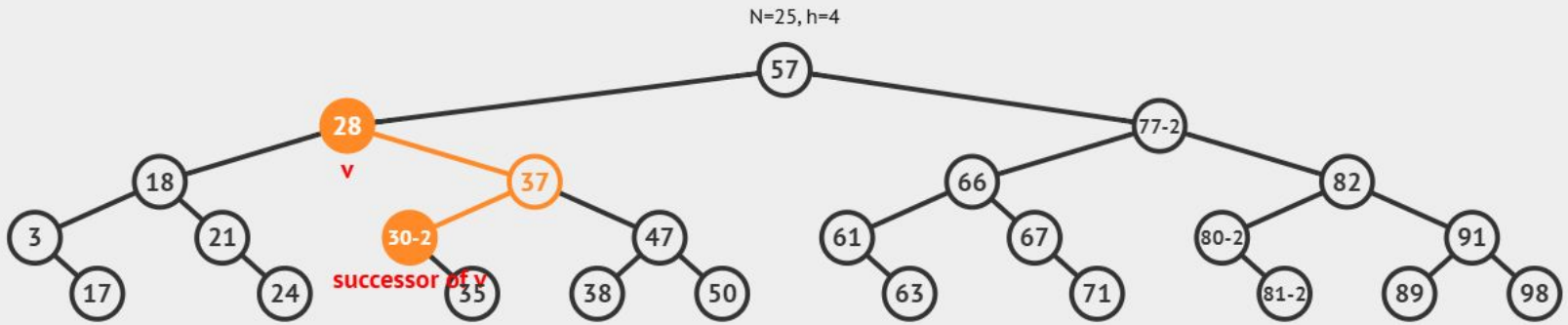
```
if this is null return empty
if left != null
  go left
else return this key
```




**Predecessor(61)**

Predecessor found!
The predecessor of 61 is 57.

```
if this.left != null return findMax(this.left)
else
  p = this.parent, T = this
  while (p != null && T == p.left)
    T = p, p = T.parent
  if p is null return -1
  else return p
```




## Successor(28)

Successor found!
The successor of 28 is 30.

```
if this.right != null return findMin(this.right)
else
  p = this.parent, T = this
  while (p != null && T == p.right)
    T = p, p = T.parent
  if p is null return -1
  else return p
```

# Conclusion

Interactive simulators are essential for demystifying recursive algorithms and pointer management in tree data structures.